Drone-Graph Architecture Report

1. Executive summary

Drone-Graph is a lightweight AI orchestration architecture for medium to complex work. It treats AI as a temporary execution unit rather than a human-like role. User goals are decomposed into atomic tasks, vetted through policy checks, routed via capabilities, executed by disposable drones, and coordinated via protocols.

Findings flow through a filtered signal system into graph-based memory, where temporary and durable knowledge are kept separate. The system is designed to complete complex work with structured coordination, capability-aware execution, and persistent graph memory, without relying on fake human organisational structures.

In Drone-Graph, AI workers are not fake CEOs, CTOs, or employees with theatrical personalities. They are **drones**: short-lived execution units spawned to complete a specific task, load the skills and tools needed for that task, return results, and terminate.

The architecture is designed for work that is too complex for a single LLM call and too fragile for loose multi-agent chaos. It sits in the middle ground between:

- a simple one-shot LLM interaction
- and a bloated human-role-based multi-agent system

Drone-Graph solves this by combining:

- **Protocol-driven coordination**
- **Graph-based memory**
- **Task decomposition**
- **Capability-based routing**
- **Temporary execution drones**
- **Signal filtering**
- **Durable and temporary knowledge separation**
- **Goal alignment checks**

The core principle is:

Protocols handle coordination. LLMs handle semantic judgment. Graphs handle memory. Drones handle execution.

That is the entire system in one line. The rest is detail, and humans do love wrapping one good idea in seventeen layers of ceremony, so let's keep the clarity.

2. Why Drone-Graph exists

2.1. The problem with simple LLM use

A plain LLM is useful for simple tasks, but it breaks down on complex work because it has several limits:

- Weak long-horizon execution
- Poor state discipline
- Fragile memory
- Difficulty tracking dependencies
- Weak provenance handling
- Tendency to mix fact, hypothesis, and guess
- Tendency to drift away from the actual goal
- No native coordination across multiple task streams

A simple model can answer, summarise, draft, and reason in one shot. But when the work becomes multi-step, tool-heavy, evidence-heavy, long-running, verification-sensitive, multi-source, iterative, a single-chat setup starts to crack.

2.2. The problem with fake agent companies

Many multi-agent systems respond to that by building a fake company with CEO agents, CTO agents, Researcher agents, Marketing agents, Operations agents etc.

That looks neat in demos and pitch decks. Underneath, it is often nonsense. If most of those workers use the same underlying model, then the titles are mostly decorative. The actual differences come from:

- Tools
- Context
- Permissions
- Constraints
- Memory access
- Cost limits
- Routing logic

So Drone-Graph rejects the fake org-chart pattern.

2.3. The core alternative

Drone-Graph says:

- Use **decomposition** instead of managerial theatre
- Use **capability routing** instead of title routing
- Use **drones** instead of fake personalities
- Use **graphs** instead of fragile chat history
- Use **protocols** for coordination
- Use **LLMs** only where semantic interpretation is required

That is the design stance.

3. System philosophy

Drone-Graph is based on the following architectural beliefs.

3.1. AI is not human

The system does not pretend that AI workers are miniature people. AI does not need Titles, Identity, Office politics, Ego, hierarchy for emotional comfort; rather, it needs a task, context, tools, constraints, memory access and an execution path

3.2. Specialisation comes from configuration, not personality

In Drone-Graph, specialisation comes from:

- Loaded skill bundles,
- Tool access,
- Backend model choice,
- Memory visibility,
- Permitted actions,
- Runtime policy

A drone is not "special" because it is named "Senior Research Lead." It is special because it was given browsing tools, retrieval access, reasoning backend, research-specific skill pack, and evidence-handling rules.

3.3. Coordination should be deterministic wherever possible

Anything that can be solved with protocols should be solved with protocols.

That includes task locking, claim ownership, retries, budgets, routing gates, state transitions, TTL rules, policy checks and event routing.

LLMs should not waste tokens on decisions that deterministic software can handle better.

3.4. Memory should be structured

The system should not store everything as one giant blob of text history. Complex work contains relationships:

- task depends on other tasks,
- finding supports claim,
- source belongs to the result,
- tool used in operation,
- A contradiction exists between two claims,
- result promoted from temporary to durable memory

That is graph-shaped, not paragraph-shaped.

3.5. Temporary work and durable knowledge are different

Not every output deserves to become permanent knowledge.

Drone-Graph separates:

a. temporary findings

b. verified durable knowledge

This avoids two equally stupid outcomes:

- losing all useful intermediate work
- storing every half-baked observation forever

4. What Drone-Graph is for

Drone-Graph is designed for **medium to complex tasks**, especially tasks that are multi-step, decomposable, evidence-heavy, iterative, long-running, multi-tool, memory-dependent, verification-sensitive, and expensive enough to justify orchestration. For example, a scientific or research workflow, large codebase analysis and repair, multi-

stage investigation, structured intelligence gathering, extended planning with verification, discovery pipelines with weak signals, long-horizon multi-source synthesis and it is **not** meant for basic chatting, simple summarisation, quick drafting, one-shot Q&A, small isolated tasks that a single good LLM can already do For simple work, a direct LLM call is better. No point rolling out a battle tank to buy milk.

5. High-level architecture overview

Drone-Graph has the following major components:

1. **User Goal Intake Layer**
2. **Decomposer**
3. **Aligner**
4. **Task Graph**
5. **Task Admission Rules**
6. **Marketplace Graph**
7. **Capability Router Protocol**
8. **Bid / Claim Protocol**
9. **Drone Execution Layer**
10. **Signal Layer**
11. **Protector / Triage Layer**
12. **Gatekeeper**
13. **Findings Graph**
14. **Sleeper**
15. **Knowledge Graph**

These components form a pipeline, but not a rigid one. The system should be **elastic**: simple tasks take a shorter path, and complex tasks use the full orchestration path.

6. End-to-end architecture flow

Here is the full end-to-end flow in plain English.

1. A user submits a goal.

2. The system interprets the goal.
3. The **Decomposer** breaks the goal into atomic tasks.
4. Those tasks are stored in the **Task Graph**.
5. Each task passes through **Task Admission Rules**.
6. Admitted tasks are sent to the **Capability Router Protocol**.
7. The router identifies the best-fit drone type or backend.
8. The **Bid / Claim Protocol** manages claiming and ownership.
9. A new drone is spawned or an eligible drone claims the task.
10. The drone queries the **Marketplace Graph** for required skills, tools, and capabilities.
11. The drone executes the task.
12. The drone emits findings and final outputs to the **Signal Layer**.
13. The **Protector / Triage Layer** filters or routes those outputs.
14. Accepted outputs go to the **Gatekeeper**.
15. The Gatekeeper decides whether the output belongs in the **Findings Graph** or the **Knowledge Graph**.
16. The **Sleeper** manages decay and retention of temporary findings.
17. The **Aligner** checks whether outputs actually satisfy the original user goal.
18. If not, new tasks may be generated, and the process continues.
19. If yes, the result is returned and the drone terminates.

That is the operational loop.

7. Core design laws

Before we describe each layer, the architecture is governed by several hard rules.

7.1. Protocols handle coordination

Anything deterministic should remain deterministic.

7.2. LLMs handle semantic judgment

Interpretation, decomposition, classification, and ambiguity resolution go to LLMs.

7.3. Drones are disposable

Drones should not become pseudo-people unless persistence is genuinely required.

7.4. The architecture must not be heavier than the task

Do not apply full orchestration to trivial work.

7.5. Every important write should carry metadata

At minimum, it should carry source, timestamp, confidence, provenance and verification status.

7.6. Temporary memory and durable memory stay separate

Intermediate work is not the same as validated knowledge.

7.7. Capability must beat confidence

A worker should not claim a task because it "feels" useful. It should be eligible by capability first.

8. Component-by-component architecture

8.1. User Goal Intake Layer

Purpose: This layer receives the user's top-level request or mission.

Responsibilities:

- Receive raw user objective
- Identify the apparent scope
- Normalise format
- Attach session, policy, and resource context
- Pass the goal to the Decomposer and Aligner

Inputs:

- user goal
- user-provided files or data
- user policies and preferences
- API keys/runtime availability
- task constraints

Outputs:

- normalized mission object
- initial goal node in the Task Graph
- context for decomposition

Why it matters: Users do not provide atomic tasks. They provide fuzzy intentions, incomplete constraints, or broad missions. This layer defines the starting boundary.

8.2. Decomposer

Purpose: Break the user goal into manageable atomic tasks.

Responsibilities:

- interpret the user goal
- identify subgoals
- break work into atomic tasks
- establish dependencies
- infer required outputs
- estimate required capabilities
- avoid under-decomposition or over-decomposition

Example:

User goal: Find promising candidates for an undiscovered planet in a dataset and rank the strongest leads.

Decomposer might create tasks such as ingest dataset, validate source integrity, identify anomalies, compare anomalies to known patterns, generate candidate hypotheses, score signal quality, check alternative explanations, summarise ranked candidates

Why it matters: Without decomposition, complex tasks become vague, untrackable blobs. The system cannot route or verify what it has not structurally defined.

LLM / protocol: Primarily **LLM-driven**, because this is a semantic task.

Failure modes:

- too coarse: task remains vague
- too fine: task graph becomes bloated
- wrong dependency ordering
- missing critical validation steps
- decomposition that optimizes convenience, not correctness

8.3. Aligner

Purpose: Ensure the system remains faithful to the original goal.

Responsibilities:

- compare intermediate and final results against user intent
- detect goal drift
- flag partial or misaligned outputs
- decide whether results are acceptable
- trigger additional work when output is incomplete

Why it matters: A multi-step system can become efficient at solving the wrong problem. The Aligner prevents local task completion from being mistaken for overall success.

LLM / protocol: Primarily **LLM-driven**, though some checks can be deterministic.

Failure modes:

- false acceptance of weak results
- over-rejection of acceptable results
- failure to detect subtle drift
- interpreting completeness too narrowly

8.4. Task Graph

Purpose: Store the execution structure of the mission.

What it stores:

- Goals
- atomic tasks
- dependencies
- state transitions
- ownership
- retries
- priorities
- blockers
- completion status
- alignment status

Why a graph?

Tasks are not flat. They depend on each other. A graph is ideal for representing parent-child task relationships, prerequisite chains, parallelizable work, blocked states, evidence linkage

Example nodes:

Mission: Discover planet candidates, Task A: Clean dataset, Task B: Detect anomalies, Task C: Validate anomaly source, Task D: Score candidate confidence

LLM/ protocol: Protocol/data structure, not LLM.

Failure modes:

- graph bloat
- stale task state
- dependency cycles
- poor task normalisation

8.5. Task Admission Rules

Purpose: Decide whether a task should be allowed into execution.

Responsibilities:

- validate that task is well-formed
- check whether required capabilities exist
- enforce security and privacy rules
- enforce cost and budget limits
- determine if the task needs approval
- reject duplicate or forbidden tasks
- constrain unsafe autonomy

Possible outputs:

- admit
- admit with constraints
- hold for approval
- reject
- escalate

Why it matters: Not every decomposed task should proceed blindly. This layer protects the system from policy violations, wasteful execution, risky action, duplicate work, and impossible tasks.

LLM/protocol: Mostly **protocol/service logic** with optional LLM assistance for ambiguous classifications.

Example:

A task requiring external web browsing may be blocked if no browsing capability exists, the privacy policy forbids external calls, or the budget is exhausted.

8.6. Marketplace Graph

Purpose: Represent available capabilities, skills, tools, and drone profiles.

What it stores:

- skill definitions
- tool interfaces
- backend model types
- drone capability profiles
- performance history
- cost profiles
- compatibility relationships
- permissions

What is a skill: A skill is not a drone. A skill is a capability that a drone can load, such as a code editing tool**, a** retrieval function**,** shell execution**,** browser research workflow**,** summarisation template**,** contradiction-checking routine**, or** graph query adapter

Why a graph: Because the relationships matter. This skill requires that tool. This backend supports that skill. This drone type performs well on that task class. This capability is incompatible with this policy. This skill bundle is optimal for certain jobs.

LLM or protocol: Mostly **structured data + protocol logic**.

Failure modes:

- vague skill definitions
- unversioned skill bundles
- no cost metadata
- capability claims not grounded in performance

8.7. Capability Router Protocol

Purpose: Select the best-fit backend or drone type for each admitted task.

Responsibilities:

- classify task type
- read required capabilities
- find eligible drones/backends
- score candidates

- apply user preferences
- apply privacy, budget, and policy constraints
- decide whether to reuse an existing drone or spawn a new one

Inputs:

- admitted task
- task metadata
- Marketplace Graph
- available API keys / local runtimes
- policies
- current load

Outputs:

- selected drone type
- selected model/backend
- selected skill bundle
- spawn or reuse decision
- fallback decision if needed

Example routing decisions:

- coding/debugging task → Claude Code / Codex-style drone
- deep research task → Opus-style research drone
- cheap classification task → lightweight low-cost drone
- privacy-sensitive task → local model if available

Why it matters: This is the component that stops the system from treating all models as interchangeable mush.

LLM or protocol: Best as a **hybrid** protocol-based eligibility and scoring, and a optional LLM help for semantic task classification

Failure modes:

- wrong capability match
- too many downgrades to weak fallback models
- ignoring policy constraints
- overfitting to cost instead of quality

8.8. Bid / Claim Protocol

Purpose: Manage task ownership and prevent execution collisions.

Responsibilities:

- task locking
- claiming
- lease ownership
- heartbeat monitoring
- timeout and reassignment
- retry count handling
- claim rejection
- status transitions

Why it matters: Without this layer, multiple drones may do the same work**,** dead drones may hold tasks forever**,** retries become chaos**, and** ownership becomes unclear.

LLM or protocol: Strictly **protocol/service logic**.

Failure modes:

- lock starvation
- orphaned tasks
- infinite retries
- no clear reclaim logic

8.9. Drone Execution Layer

Purpose: Perform the actual work.

What is a drone?

A drone is a temporary execution unit composed of:

- a task
- a backend model
- a selected skill bundle
- allowed tools
- permissions
- temporary working memory
- budget and runtime constraints

Responsibilities:

- accept or receive task
- load relevant skills
- access required context

- execute work
- emit findings
- return final output
- terminate

Why drones are temporary: Because if all generic workers use the same underlying intelligence, then persistent identity is mostly unnecessary. The specialisation is in configuration, not personality.

When drones should persist: Only in special cases, such as long-running watchers**,** continuous monitoring jobs**,** persistent subscriptions, and durable external sessions

LLM or protocol: The drone's reasoning is **LLM-driven**, but its lifecycle is protocol-controlled.

Failure modes:

- overclaiming tasks outside one's capability
- poor skill loading
- output without provenance
- excessive autonomy
- weak error recovery

8.10. Signal Layer

Purpose: Move findings and events through the system.

Responsibilities:

- accept findings from drones
- structure them as events
- broadcast only where relevant
- support subscription-based listening
- separate event types

Example event types: *task_claimed***,** *task_failed***,** *blocker_found***,** *evidence_found***,** *contradiction_detected, result_ready***,** *verification_required*

Why it matters: This avoids constant polling and enables responsive coordination.

Best design: Use typed channels and subscriptions, not one giant everything-to-everyone broadcast stream.

LLM or protocol: Protocol/message system.

Failure modes:

- signal overload
- irrelevant broadcast noise
- no event prioritization
- delayed delivery

8.11. Protector / Triage Layer

Purpose: Filter and route findings before they contaminate the wider system.

Responsibilities:

- check relevance
- check confidence
- check provenance completeness
- flag contradictions
- detect low-quality or suspicious outputs
- quarantine or escalate findings

Important clarification: The Protector should not pretend to be an omniscient truth god. Its job is not "determine truth forever." Its job is to score, route, quarantine, prioritize and escalate for verification

Why it matters: If every finding spreads freely, the system becomes a hallucination amplifier.

LLM or protocol: Best as a **hybrid** with a rule-based check for metadata, provenance, duplication and an LLM assistance for semantic inconsistency or ambiguity.

Failure modes:

- overblocking useful work
- underblocking junk
- bottlenecking the signal system
- binary truth decisions on ambiguous evidence

8.12. Gatekeeper

Purpose: Place accepted knowledge in the right memory structure.

Responsibilities:

- decide where new information belongs
- classify output as temporary, durable, or rejected
- attach provenance

- assign confidence and freshness
- validate structural placement
- trigger verification when needed

Why it matters: One of the hardest problems in complex AI systems is not producing information, but deciding where it belongs, whether it should persist, and how trusted it should be.

Typical outcomes:

- send to Findings Graph
- send to Knowledge Graph
- quarantine
- reject
- queue for verification

LLM or protocol: Best as a **hybrid**.

Failure modes:

- promoting noise to durable knowledge
- losing useful temporary findings
- wrong graph placement
- weak provenance linking

8.13. Findings Graph

Purpose: Store temporary or unverified work.

What it stores:

- intermediate outputs
- raw observations
- partial conclusions
- hypotheses
- recent evidence
- weak signals
- pending verification items

Why it matters: Temporary findings are often useful, but not durable enough to become system truth.

Difference from Knowledge Graph: Findings Graph is temporary, exploratory, lower-trust, short-lived, active-context-oriented

Failure modes:

- excessive growth
- poor retention criteria
- contamination with permanent facts
- losing traceability

8.14. Sleeper

Purpose: Manage the lifecycle of temporary findings.

Responsibilities:

- apply TTL rules
- prune stale findings
- compress redundant findings
- archive useful but inactive data
- remove contradicted or dead-end material
- manage depth and retention boundaries

Why it matters: Without this layer, the Findings Graph becomes a junkyard.

Retention logic examples:

- remove low-confidence findings after 48 hours
- retain findings linked to active tasks
- compress repeated similar signals
- archive high-value but inactive evidence
- delete contradicted weak findings

Failure mode:

- deleting valuable material too early
- retaining garbage too long
- no archive path
- Retention rules are too vague

LLM or protocol: Mostly **protocol/service logic**, with optional LLM assistance for semantic clustering/compression.

8.15. Knowledge Graph

Purpose: Store durable, validated, system-level knowledge.

What it stores:

- verified facts
- entities
- relations
- validated conclusions
- capabilities
- tools
- decisions
- historical provenance
- confidence and freshness metadata

Why it matters: This is long-term memory, but structured.

Minimum metadata per knowledge item:

- source
- timestamp
- confidence
- provenance chain
- verification status
- freshness/staleness state

Failure modes:

- promoting temporary noise
- stale facts never revalidated
- missing contradiction tracking
- weak evidence stored as permanent truth

9. Architecture roles: LLMs vs. protocols

9.1. LLM-driven

Decomposer, Aligner, semantic task classification, ambiguous verification, contradiction analysis and nuanced knowledge classification.

9.2. Protocol-driven

task claiming, ownership, retries, budgets, admission checks, state transitions, TTL and pruning, event routing, permissions and schema enforcement.

9.3. Hybrid

Capability Router**,** Protector**,** Gatekeeper**,** contradiction handling**,** quality triage

This split prevents the system from becoming an expensive bureaucracy of models doing what ordinary software should have done in 3 milliseconds.

10. Data model: the three-graph memory design

Drone-Graph works best when memory is separated into three distinct graph spaces.

10.1. Task Graph

Execution memory **Stores goals, tasks, dependencies, ownership, state, retries and blockers.**

10.2. Findings Graph

Temporary exploratory memory Stores **hypotheses, raw findings, partial evidence, recent discoveries and unresolved contradictions.**

10.3. Knowledge Graph

Durable, validated memory stores verified facts, decisions, entities, relationships, durable conclusions and trusted system knowledge.

11. Drone lifecycle

A drone should usually follow this lifecycle:

1. spawned
2. assigned task
3. loads context
4. queries Marketplace Graph
5. acquires skill bundle
6. executes
7. emits findings/events
8. returns final result

9. terminates

Optional states: blocked, retrying, escalated and persistent monitoring mode.

Why termination matters: Termination keeps the system honest, no fake personalities, no identity inflation, less memory contamination, better isolation and cleaner debugging.

12. Capability routing in more detail

Capability routing is one of the most important parts of the architecture.

12.1. Inputs to routing

- task type
- required tools
- required skill bundle
- output type
- privacy constraints
- budget sensitivity
- urgency
- user preferences
- backend availability

12.2. Candidate scoring dimensions

- capability fit
- model quality for task class
- tool support
- cost
- latency
- trust/reliability history
- permission compatibility
- local vs external preference

12.3. Routing outputs

- existing drone reuse
- spawn new specialist drone
- use default drone
- reject or escalate

Routing rule: Eligibility first, claiming second**.** A drone should not be allowed to claim a task until the router has determined it is actually fit for the job.

13. Task admission in more detail

Task Admission Rules act as the first serious control gate.

13.1. Questions admission should answer

a. Is this task valid?
b. Is it already done?
c. Is it forbidden?
d. Is it redundant?
e. Does it require unavailable capabilities?
f. Is it within budget?
g. Is it too risky for auto-execution?
h. Does it require human approval?
i. Does it violate privacy policy?

13.2. Why this layer is essential

Without admission control, the system wastes resources or performs unsafe actions just because decomposition produced a task node. A task's existence is not proof that it deserves execution.

14. Signal and findings flow

Signal flow is how Drone-Graph avoids worker isolation without becoming noisy chaos.

14.1. Signal architecture:

typed events, subscriptions, relevance filtering**,** priority levels and provenance metadata.

14.2. Things to avoid:

Everything broadcast to everything, no filtering, no event types, no confidence markers, no contradiction tracking

The Protector need to exist because a raw signal layer alone is not enough.

15. How the system completes a complex mission

Let's use a hypothetical mission:

Identify possible new planet candidates in a Milky Way observational dataset

Step 1: Intake

User submits mission, data, and constraints.

Step 2: Decomposition

Tasks created:

- validate dataset
- identify anomalies
- cluster patterns
- compare to known planet indicators
- test alternative explanations
- score candidates
- summarise best leads

Step 3: Admission

Each task is checked for:

- scientific workflow validity
- tooling availability
- compute budget
- allowed autonomy

Step 4: Routing

Tasks are routed

- anomaly analysis → data-analysis drone
- literature comparison → research drone
- scoring → reasoning-heavy drone

Step 5: Execution

Drones loads relevant skills:

- statistical tool
- retrieval workflow
- scoring rubric
- graph query adapter

Step 6: Signal emission

Findings emerge:

- anomaly X detected

- source calibration issue found
- known false-positive pattern matched
- candidate group ranked

Step 7: Protector

Weak or unsupported findings are quarantined. Strong findings are allowed through.

Step 8: Gatekeeper

- raw anomaly note → Findings Graph
- validated candidate score → Knowledge Graph
- unsupported speculation → quarantine

Step 9: Sleeper

Old weak signals expire. High-value temporary findings remain until the mission ends.

Step 10: Alignment

Aligner checks:

- did the system produce actual ranked candidates?
- were alternative explanations considered?
- does the output match the mission?

If not, new tasks are created. This is how Drone-Graph gets real work done.

16. Main risks

The architecture is good, but it can still fail. The main risks are:

16.1. Overengineering

If every small task activates the whole stack, the system becomes slower than useful.

16.2. Misrouting

If capability routing is weak, the wrong backend handles the work.

16.3. Weak provenance

If findings lack evidence metadata, memory becomes polluted.

16.4. Graph bloat

If everything is stored forever, performance and clarity collapse.

16.5. Over-trusting LLM triage

If the Protector or Gatekeeper behave like a fake oracle, hallucinations will be formalised.

16.6. Poor decomposition

If decomposition is wrong, the rest of the pipeline only executes the bad structure more efficiently.

16.7. Missing uncertainty tracking

For discovery or research tasks, a lack of uncertainty modelling is fatal.

17. Drone-Graph working conditions

For the system to actually "get the work done," these conditions must hold:

1. **Simple tasks must have a thin path**
2. **Complex tasks must justify orchestration**
3. **Routing must be capability-aware**
4. **Drones must remain disposable by default**
5. **Graphs must carry strong metadata**
6. **Findings and knowledge must stay separate**
7. **Protocols must dominate coordination**
8. **LLMs must only be used where semantic ambiguity exists**
9. **The system must track uncertainty**
10. **Every persistent layer must solve a real failure mode**

That last one matters a lot. If a box exists only because it looks sophisticated, kill it.

18. Conclusion

Drone-Graph is a coherent architecture that makes the right separations between coordination vs reasoning, execution vs identity, temporary vs durable memory, capability vs confidence, graph state vs chat history, simple path vs complex path

The architecture will fail if it becomes too LLM-heavy or too eager to preserve every output as truth. In other words, it will fail the same way humans usually fail: by turning a sharp system into a bloated institution.

Drone-Graph is best understood as a protocol-governed, graph-native, capability-routed AI execution system built around disposable drones.